QUANTITATIVE BIOLOGY LAB

UNIVERSITÀ DI TORINO

di.unito.it
DIPARTIMENTO DI INFORMATICA

Young InfoLife
cini national lab

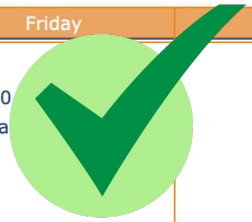
From Data to Models:
Analyzing and Integrating Biological Data into Mechanistic Models

Dora Tortarolo, Simone Pernice, Francesca Cordero

Course of *Doctoral school*
PhD in Complex Systems for Quantitative Biomedicine

When?

Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
14	15	16	17	18	• 09:00 Semina	20
21	22 • 09:00 - 13:00 Sala conferenze, terzo piano	23	24	25	26 • 09:00 - 13:00 Sala conferenze, terzo piano	27
28	29 • 09:00 - 13:00 Sala conferenze, terzo piano	30	1	2	3	4



Course Outline

Day 2 – Calibration and Predictive Modeling

- In-Depth Model Creation
- Model Calibration: introduction to calibration techniques for mechanistic models.
- Predictive Simulations: Performing what-if analyses using calibrated models.

https://github.com/qBioTurin/From-Data-to-Models.git

How Can We Simulate the Dynamics of a Real System?

Given **real data** or **biological knowledge**, how can we set a model that represents system dynamics?

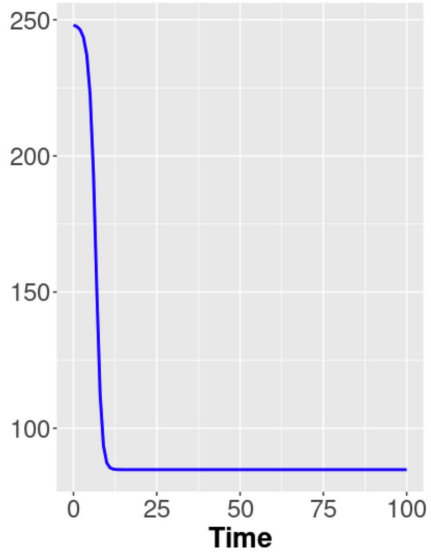
Model calibration refers to the process of adjusting parameters in a mathematical model so that the model's output matches observed data as closely as possible.

Model Calibration

An **ODE model** describes how a system evolves over time using differential equations. Mathematically:

$$\frac{dy(t)}{dt} = f(y(t), \theta, t)dt$$

Solving (integrating) the equations



- *y(t)*: state variables (e.g., concentration)
- *θ*: parameters (e.g., growth rate, decay constant) — these are what we calibrate
- *f()*: the model (defines the dynamics of the system)
- *t*: time

Model Calibration

An **ODE model** describes how a system evolves over time using differential equations. Mathematically:

$$\frac{dy(t)}{dt} = f(y(t), \theta, t)dt$$

- *y(t)*: state variables (e.g., concentration)
- *θ*: parameters (e.g., growth rate, decay constant) — these are what we calibrate
- *f()*: the model (defines the dynamics of the system)
- *t*: time

The same explanation can be done also for stochastic process, but for simplicity we will consider mainly the deterministic process.

Model Calibration

Calibration ⟹ **Parameter Estimation**

Given:

- An ODE model: $\frac{dy(t)}{dt} = f(y(t), \theta, t)dt$
- Observed data: $y_{\text{obs}}(t_i)$, at discrete time points $t_1, t_2, \ldots, t_n$

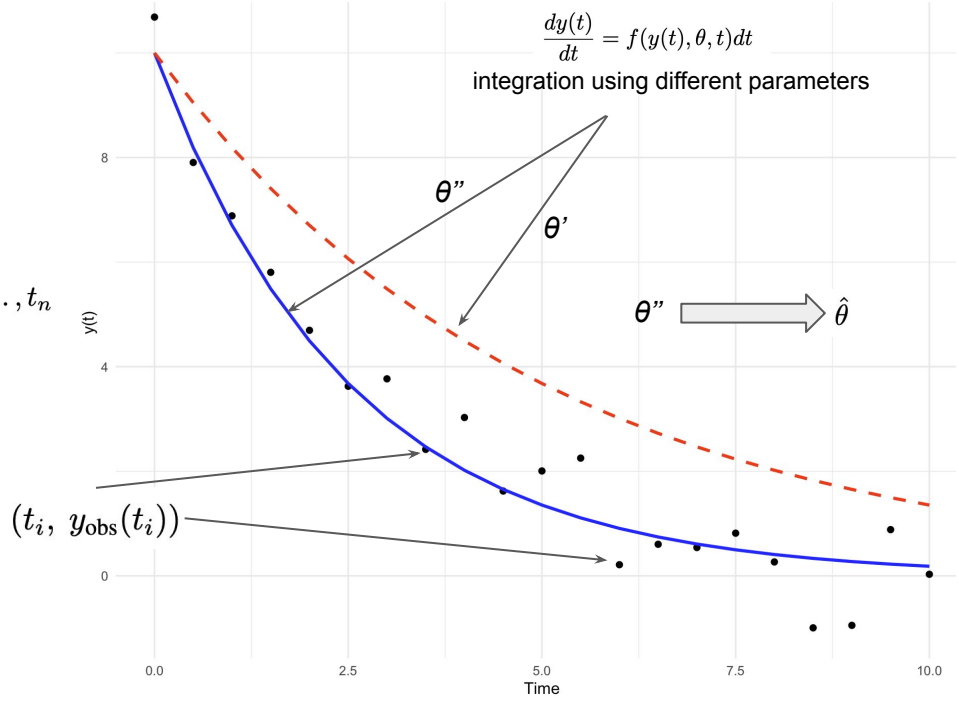
You want to find the **best parameter values** $\hat{\theta}$ such that the **simulated model output** $\hat{y}(t_i; \hat{\theta})$ matches the observed data

Model Calibration

Calibration ⇨ **Parameter Estimation**

Given:

- An ODE model: $\frac{dy(t)}{dt} = f(y(t), \theta, t)dt$
- Observed data: $y_{\text{obs}}(t_i)$, at discrete time points $t_1, t_2, \ldots, t_n$



You want to find the **best parameter values** $\hat{\theta}$ such that the **simulated model output** $\hat{y}(t_i; \hat{\theta})$ matches the observed data

Model Calibration

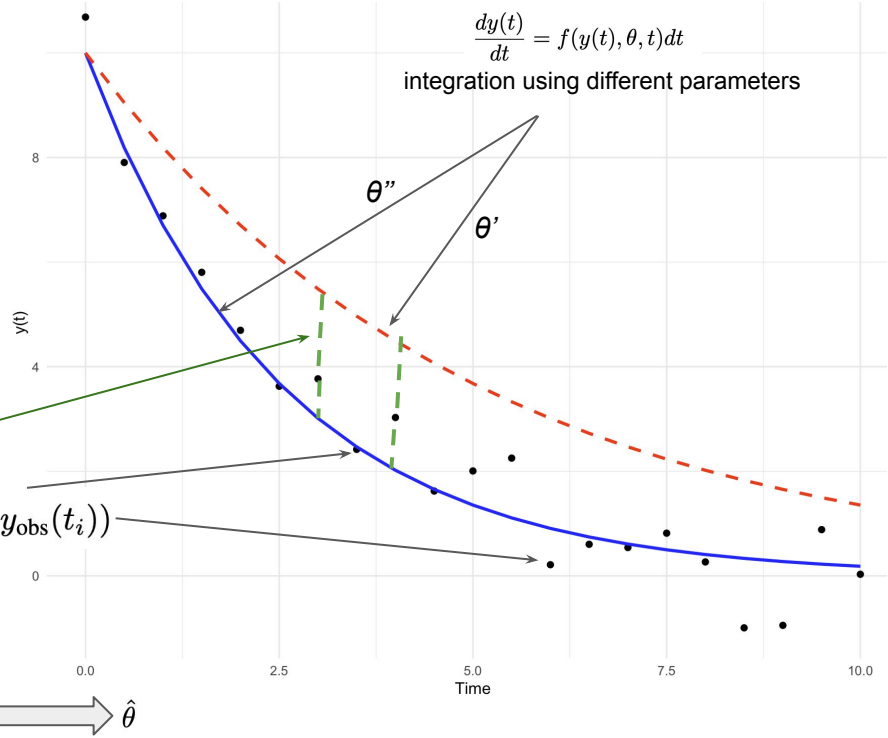
Calibration ⟹ **Parameter Estimation**

Given:

- An ODE model: $\frac{dy(t)}{dt} = f(y(t), \theta, t)dt$
- Observed data: $y_{\text{obs}}(t_i)$, at discrete time points $t_1, t_2, \ldots, t_n$

Where D is a distance (or loss) function measuring the difference between model and data.

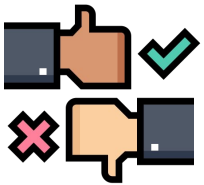
$$\hat{\theta} = argmin_{\theta} D(y_{\text{obs}}(t), \hat{y}(t; \hat{\theta}))$$



You want to find the **best parameter values** $\hat{\theta}$ such that the **simulated model output** $\hat{y}(t_i; \hat{\theta})$ matches the observed data

Model Calibration: distance

Metric	Expression
Squared Error (L2 norm)	$\sum_{i=1}^{n} \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$

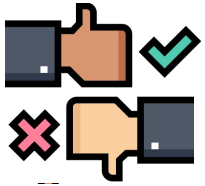


Simple, efficient, widely supported.

Sensitive to outliers — a few large errors can dominate

Model Calibration: distance

Metric	Expression
Squared Error (L2 norm)	$\sum_{i=1}^{n} \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Absolute Error (L1 norm)	$\sum_{i=1}^{n} \left\lvert y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right\rvert$

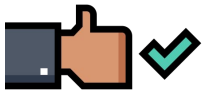


More robust when data contains outliers or noise

Not differentiable everywhere; optimization may be slower.

Model Calibration: distance

Metric	Expression
Squared Error (L2 norm)	$\sum_{i=1}^{n} \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Absolute Error (L1 norm)	$\sum_{i=1}^{n} \left\lvert y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right\rvert$
Weighted Least Squares	$\sum_{i=1}^{n} w_i \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$



Assigns **different importance** (weights wi) to different time points.

Useful when some data points are more **reliable** or **important**

You trust early-time measurements more than later ones. Set higher wi at the beginning.

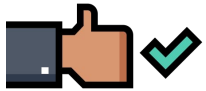
Choosing appropriate weights may be subjective.

Model Calibration: distance

Metric	Expression
Squared Error (L2 norm)	$\sum_{i=1}^{n} \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Absolute Error (L1 norm)	$\sum_{i=1}^{n} \left\lvert y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right\rvert$
Weighted Least Squares	$\sum_{i=1}^{n} w_i \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Mahalanobis Distance	$\sum_{i=1}^{n} (y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i;\theta))^T \Sigma_i^{-1} (y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i;\theta))$

covariance matrix of the data at time t_i

You're capturing **correlations between outputs**, not across time



Accounts for **correlation and variance** among output variables.

Often used in **multivariate models**.

If two model outputs are strongly correlated (e.g., mRNA and protein levels), Mahalanobis avoids double-counting that information.

Requires estimating or knowing Σ.

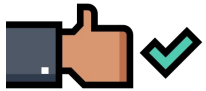
Model Calibration: distance

Metric	Expression
Squared Error (L2 norm)	$\sum_{i=1}^{n} \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Absolute Error (L1 norm)	$\sum_{i=1}^{n} \lvert y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\rvert$
Weighted Least Squares	$\sum_{i=1}^{n} w_i \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Mahalanobis Distance	$(y_{\text{model}} - y_{\text{obs}})^T \Sigma^{-1} (y_{\text{model}} - y_{\text{obs}})$

covariance matrix of the data

You treat each full time series as a long vector

$$\mathbf{y}_{\text{obs}} = \begin{bmatrix} y_1(t_1) \\ y_1(t_2) \\ \vdots \\ y_2(t_1) \\ \vdots \end{bmatrix}, \quad \mathbf{y}_{\text{model}} = \begin{bmatrix} \hat{y}_1(t_1) \\ \hat{y}_1(t_2) \\ \vdots \\ \hat{y}_2(t_1) \\ \vdots \end{bmatrix}$$



Accounts for **correlation and variance** among output variables.

Requires a **large enough dataset** to estimate the full covariance matrix across all times

Model Calibration: distance

Metric	Expression
Squared Error (L2 norm)	$\sum_{i=1}^{n} \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Absolute Error (L1 norm)	$\sum_{i=1}^{n} \lvert y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\rvert$
Weighted Least Squares	$\sum_{i=1}^{n} w_i \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Mahalanobis Distance	$(y_{\text{model}} - y_{\text{obs}})^T \Sigma^{-1} (y_{\text{model}} - y_{\text{obs}})$
Relative Error	$\sum_{i=1}^{n} \left(\frac{y_{\text{model}}\ (t_i;\theta) - y_{\text{obs}}\ (t_i)}{y_{\text{obs}}\ (t_i)} \right)^2$

Relative error compares model predictions to observations **as a proportion** of the observation

Even though the absolute error in time 1 is larger, the **relative error is smaller**, which makes this method fair across scales.

Model Calibration: distance

Metric	Expression
Squared Error (L2 norm)	$\sum_{i=1}^{n} \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Absolute Error (L1 norm)	$\sum_{i=1}^{n} \left\lvert y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right\rvert$
Weighted Least Squares	$\sum_{i=1}^{n} w_i \left(y_{\text{model}}(t_i;\theta) - y_{\text{obs}}(t_i)\right)^2$
Mahalanobis Distance	$(y_{\text{model}} - y_{\text{obs}})^T \Sigma^{-1} (y_{\text{model}} - y_{\text{obs}})$
Relative Error	$\sum_{i=1}^{n} \left(\frac{y_{\text{model}}\ (t_i;\theta) - y_{\text{obs}}\ (t_i)}{y_{\text{obs}}\ (t_i)} \right)^2$

Average-Based Objective Functions

$\text{MRE} = \frac{1}{n}\sum_{i=1}^{n} \left\lvert \frac{y_{\text{model}}\ (t_i) - y_{\text{obs}}\ (t_i)}{y_{\text{obs}}\ (t_i)} \right\rvert$	Mean Relative Error	Gives the **average % error per point**
$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} \lvert y_{\text{model}}\ (t_i) - y_{\text{obs}}\ (t_i)\rvert$	Mean Absolute Error	Purely based on magnitudes; Still sensitive to variable scales
$\text{MSRE} = \frac{1}{n}\sum_{i=1}^{n} \left(\frac{y_{\text{model}}\ (t_i) - y_{\text{obs}}\ (t_i)}{y_{\text{obs}}\ (t_i)} \right)^2$	Mean Squared Relative Error	Penalizes larger relative errors more

Model Calibration: distance

To solve the **parameter estimation problem**:

$$\hat{\theta} = argmin_{\theta} D(\ y_{\text{obs}}(t), \hat{y}(t;\hat{\theta})\ )$$

where D(θ) is a **distance (or error)** between the model output (with parameters θ) and observed data, you can use **optimization algorithms**.

The right choice depends on the properties of D — e.g., whether it's differentiable, noisy, or expensive to evaluate.

Model Calibration: distance

Method	Type	Gradient Needed?	Suitable For	Notes
Gradient Descent	Local, gradient	✅ Yes	Smooth and differentiable D	Fast but gets stuck in local minima
Levenberg-Marquardt	Local, gradient	✅ Yes (or approx)	Least squares (quadratic-like)	Often used for ODE fitting
Simulated Annealing	Global, stochastic	❌ No	Complex landscapes with many minima	Slow but robust
Genetic Algorithms	Global, stochastic	❌ No	Large search space, black-box D	Parallelizable

The most used\famous optimization algorithms, but there are more….

Gradient Descent - batch gradient descent

Gradient Descent is an **iterative optimization algorithm** that updates parameters in the direction of the **negative gradient** of the objective function:

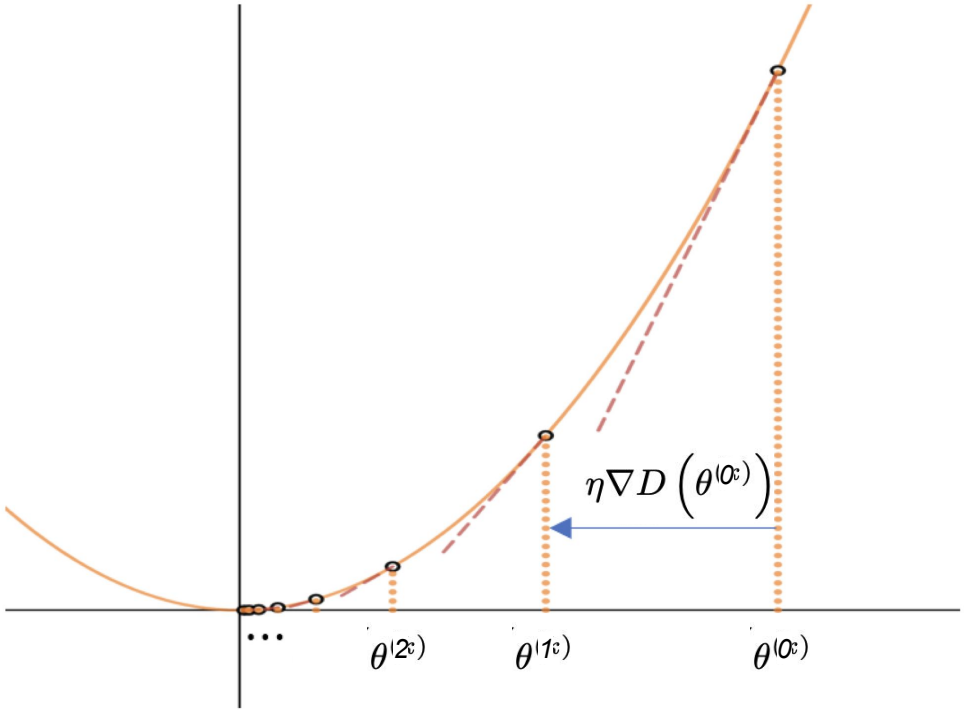
$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla D\left(\theta^{(k)}\right)$$

the parameters at iteration k, k+1

the learning rate (step size)

gradient of the distance function

Theorem: *The gradient descent algorithm converges to the global minimum of a convex loss function.*



D(θ) is differentiable and relatively smooth

Can **get stuck in local minima**

Requires **tuning** of learning rate

Sensitive to **initial guess**

Gradient Descent - batch gradient descent

Gradient Descent is an **iterative optimization algorithm** that updates parameters in the direction of the **negative gradient** of the objective function:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla D\left(\theta^{(k)}\right)$$

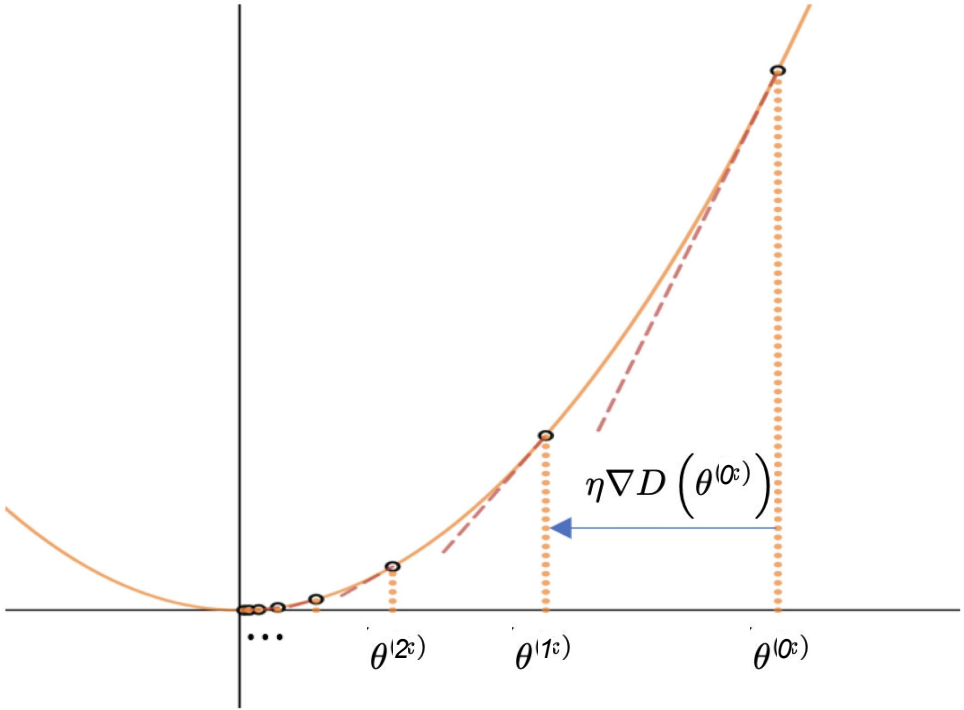
the parameters at iteration k, k+1

the learning rate (step size)

gradient of the distance function

Improvement 1: Scheduling learning rate

For example, we can start with a large learning rate to make quick progress in the beginning, and then gradually decrease it as we get closer to the minimum. This can help prevent overshooting and improve convergence.



Gradient Descent - batch gradient descent

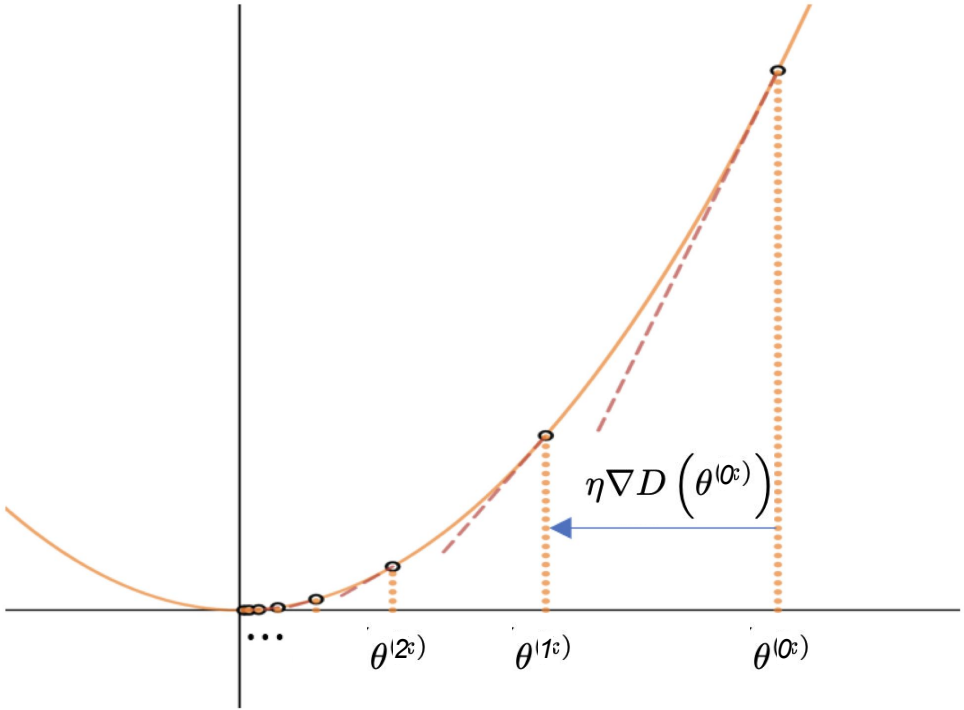
Gradient Descent is an **iterative optimization algorithm** that updates parameters in the direction of the **negative gradient** of the objective function:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla D\left(\theta^{(k)}\right)$$

the parameters at iteration k, k+1

the learning rate (step size)

gradient of the distance function



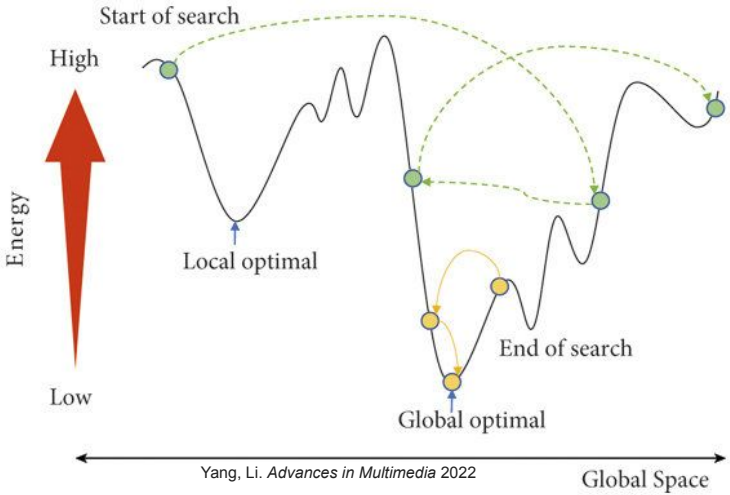
Improvement 2: Adaptive learning rates

Adjusting the learning rate based on the magnitude of the gradient. If the gradient is large, we may want to take smaller steps to prevent overshooting. If it is small, we may want to take larger steps to speed up convergence.

Simulated Annealing

Global stochastic optimization method inspired by the **cooling of metals**:

- Starts with a high "temperature" to allow **big moves**, including **worse** ones
- Gradually reduces temperature to **refine** solution



Yang, Li. *Advances in Multimedia* 2022

It is inspired by metallurgy, where a metal is heated to high temperatures and slowly cooled to minimize structural defects and reach a stable configuration!

Aarts E. and Korst J., Simulated Annealing and Boltzmann Machines, John Wiley & Sons, 1990

Simulated Annealing

The algorithm starts with an initial solution and a high "temperature," which gradually decreases over time.

1. ***Initialization***: Begin with an initial solution θ and an initial temperature T0. The temperature controls how likely the algorithm is to accept worse solutions as it explores the search space

2. ***Neighborhood Search***: At each step, a new solution θ′ is generated by making a small change (or perturbation) to the current solution θ.

3. ***Objective Function Evaluation***: The new solution θ' is evaluated using the objective function. If θ' provides a better solution than θ, it is accepted as the new solution.

4. ***Acceptance Probability***: If θ' is worse than θ, it may still be accepted with a probability based on the temperature and the difference in objective function values. The acceptance probability is given by:

$$P = \exp\left(-\frac{D(\theta') - D(\theta)}{T}\right)$$

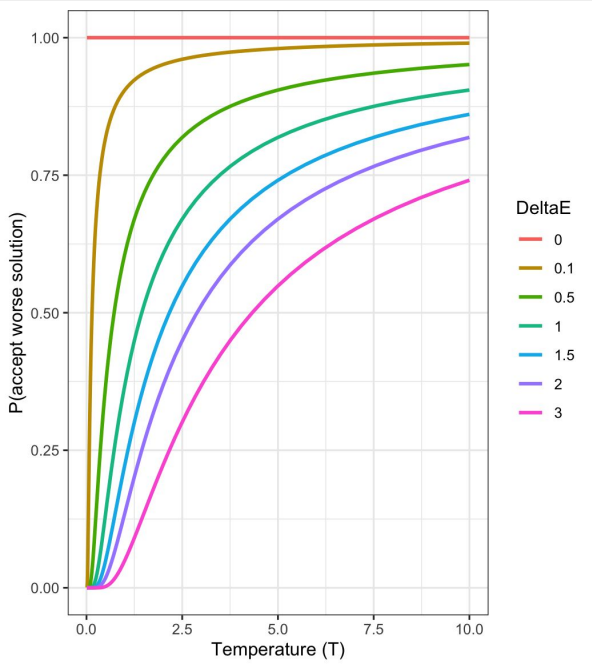
5. ***Cooling Schedule***: After each iteration, the temperature is decreased according to a predefined cooling schedule, which determines how quickly the algorithm converges.

6. ***Termination***: The algorithm continues until the system reaches a low temperature (i.e., no more significant improvements are found), or a predetermined number of iterations is reached.

Simulated Annealing

1. ***Acceptance Probability***: If θ' is worse than θ, it may still be accepted with a probability based on the temperature and the difference in objective function values. The acceptance probability is given by:

$$P = \exp\left(-\frac{D(\theta') - D(\theta)}{T}\right)$$

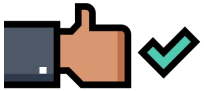


- At low temperatures, even small increases in error are very unlikely to be accepted.
- At high temperatures, worse solutions are accepted more often — helping escape local minima.

Simulated Annealing

Global stochastic optimization method inspired by the **cooling of metals**:

- Starts with a high "temperature" to allow **big moves**, including **worse** one
- Gradually reduces temperature to **refine** solution



Distance function is **non-smooth**, **noisy**, or has **many local minima**

Limitations of Simulated Annealing

- The performance of Simulated Annealing is highly dependent on the choice of parameters, particularly the initial temperature and cooling schedule.
- Since Simulated Annealing requires many iterations, it can be computationally expensive, especially for large problems.
- The convergence rate is generally slower than more deterministic methods like gradient-based optimization.

Simulated Annealing vs. Gradient Descent: Gradient descent is faster but can easily get stuck in local minima. Simulated Annealing, on the other hand, can escape local minima but is generally slower.

Genetic Algorithms

Inspired by **biological evolution** (on the ideas of natural selection and genetics):

- Simulate “survival of the fittest” among individuals of consecutive generations to solve a problem.

- The population in a genetic algorithm is a set of candidate solutions, often called individuals (θ as "individuals"). Each individual is represented as a chromosome, which can be coded as binary strings, arrays, or other data structures. For example, each chromosome could represent a set of input values to a function we need to optimize.

- The selection function chooses individuals from the population to reproduce based on their fitness. Individuals with higher fitness are more likely to be chosen for reproduction. This mimics natural selection, where the best-adapted individuals are more likely to survive and reproduce.

- Use operators like selection, crossover, and mutation to combine information and to introduce random changes.

Genetic Algorithms

Step 1: Initialization We generate an initial population of random individuals. This step creates a diverse set of potential solutions to start the algorithm.

Step 2: Evaluation Next, we need to calculate the fitness of each individual in the population. Here we use the fitness function to evaluate how good each solution is.

Step 3: Selection Using the selection criteria, we select individuals for reproduction based on their fitness. This step determines which individuals will be parents.

Step 4: Crossover Crossover comes next. By combining the genetic material of selected parents, we apply crossover techniques to generate new solutions or offspring.

Step 5: Mutation To maintain diversity in our population, we need to introduce random mutations in the offspring.

Step 6: Replacement We next replace some or all of the old population with the new offspring, by determining which individuals move on to the next generation.

Step 7: Repeat The previous steps 2-6 are looped over for a set number of generations or until a termination condition is met. This loop allows the population to evolve over time, hopefully resulting in a good solution.

Genetic Algorithms

Why use Genetic Algorithms

- They are Robust
- Provide optimisation over large space state.

Limitations

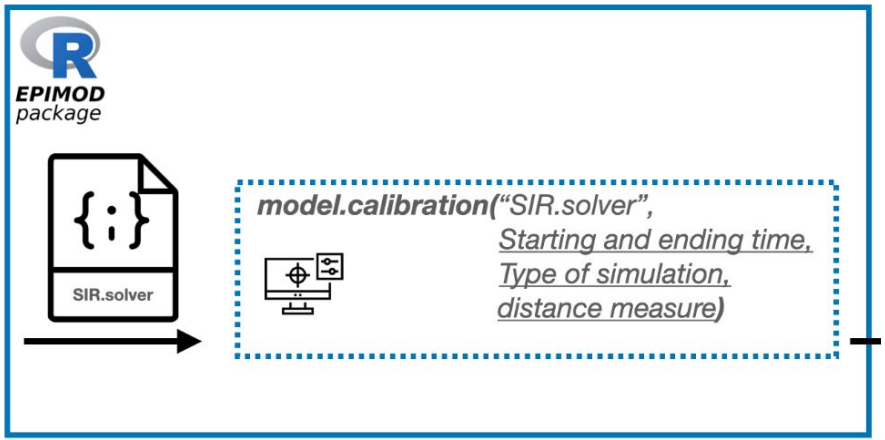
- Computationally expensive
- Many parameters to tune: mutation rate, population size, etc.
- No guarantee of global optimum

Simulated Annealing vs. Genetic Algorithms: While both methods are probabilistic and capable of escaping local minima, GAs use a population of solutions and evolve them over generations, whereas SA works with a single solution that is iteratively improved.

Calibration in R

GenSA: R Functions for Generalized Simulated Annealing

Performs search for global minimum of a very complex non-linear objective function with a very large number of optima.



https://cran.r-project.org/web/packages/GenSA/index.html

Case study: Shlogel model

1. $A + 2X \xrightarrow{k_1} 3X$
2. $3X \xrightarrow{k_2} A + 2X$
3. $B \xrightarrow{k_3} X$
4. $X \xrightarrow{k_4} B$

Sensitivity analysis

“the study of how uncertainty in the output of a model (numerical or otherwise) can be apportioned to different sources of uncertainty in the model input”

- Parameters often represent real physical/biological quantities
- Many are uncertain, estimated, or noisy

- Identify critical parameters
- Detect robust vs fragile behavior
- Guide parameter estimation and data collection
- Simplify models (remove insensitive parameters)

Andrea Saltelli, Stefano Tarantola, Francesca Campolongo, and Marco Ratto. *Sensitivity analysis in practice: a guide to assessing scientific models*. Volume 1. Wiley Online Library, 2004.

Sensitivity analysis

Local sensitivity analysis (LSA) evaluates model output changes around a specific input point (base case), usually via partial derivatives, making it efficient for simple, fast analysis.

Global sensitivity analysis (GSA) assesses how input uncertainties affect output over the entire input space, mapping parameter interactions.

Simple Example

Imagine you're planning a road trip.
Some things are fixed, like the distance. Others are uncertain, like traffic and gas prices.

You might ask:

- What happens to the total cost if gas prices increase?
- What happens to travel time if traffic gets worse?

By changing one factor at a time and seeing how the result changes, you understand what really matters for your trip.
This is exactly what sensitivity analysis does:

it studies how changes in inputs (like traffic or gas price) affect the output (total time or cost).

Even if you don't know the exact conditions in advance, this helps you make better decisions.

https://thedecisionlab.com/reference-guide/statistics/sensitivity-analysis

Some Initial Definitions

Variables: Factors (input parameters) within a model that can change. Independent variables are intentionally manipulated in an experiment or model to observe their impact on dependent variables.

X_1 : gas price

X_2 : traffic

Output: The result or outcome of a model that changes in response to alterations in independent variables.

Y: total time

Sample: A **sample** is one specific combination of input parameter values, together with the corresponding model output.

Sensitivity analysis

If I vary one parameter, while accounting for the others, how strongly is it (*monotonically) related* to the output?

How much of the output variability is caused by each parameter (and their interactions)?

Pearson Correlation Coefficient (PEARCC)

The Pearson correlation coefficient $\rho_{X,Y}$ is one of several correlation coefficients that you need to choose between when you want to measure a correlation.

The Pearson correlation coefficient is a good choice when **all** of the following are true:

- **Two Variables:** Measures the linear relationship between two variables X and Y
- **Both variables are quantitative:** You will need to use a different method if either of the variables is qualitative.
- **The variables are normally distributed:** You can create a histogram of each variable to verify whether the distributions are approximately normal. It's not a problem if the variables are a little non-normal.
- **The data have no outliers:** Outliers are observations that don't follow the same patterns as the rest of the data. A scatterplot is one way to check for outliers—look for points that are far away from the others.
- **The relationship is linear:** "Linear" means that the relationship between the two variables can be described reasonably well by a straight line. You can use a scatterplot to check whether the relationship between two variables is linear.

Pearson Correlation Coefficient (PEARCC)

The Pearson correlation coefficient $\rho_{X,Y}$ is one of several correlation coefficients that you need to choose between when you want to measure a correlation.

Idea: Measure how two variables move together (linearly).

$$\rho_{X,Y} = \frac{\operatorname{Cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]}{\sqrt{\mathbb{E}\left[X^2\right] - (\mathbb{E}[X])^2}\sqrt{\mathbb{E}\left[Y^2\right] - (\mathbb{E}[Y])^2}}$$

$$\mu_X = \mathbb{E}[X]$$

$$\mu_Y = \mathbb{E}[Y]$$

$$\sigma_X^2 = \mathbb{E}\left[(X - \mathbb{E}[X])^2\right] = \mathbb{E}\left[X^2\right] - (\mathbb{E}[X])^2$$

$$\sigma_Y^2 = \mathbb{E}\left[(Y - \mathbb{E}[Y])^2\right] = \mathbb{E}\left[Y^2\right] - (\mathbb{E}[Y])^2$$

$$\operatorname{cov}(X,Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)] = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y],$$

Pearson Correlation Coefficient (PEARCC)

The Pearson correlation coefficient $\rho_{X,Y}$ is one of several correlation coefficients that you need to choose between when you want to measure a correlation.

Idea: Measure how two variables move together (linearly).

$$\rho_{X,Y} = \frac{\operatorname{Cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]}{\sqrt{\mathbb{E}\left[X^2\right] - (\mathbb{E}[X])^2}\sqrt{\mathbb{E}\left[Y^2\right] - (\mathbb{E}[Y])^2}}$$

Given paired data $\{(x_1, y_1), \ldots, (x_n, y_n)\}$

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

Pearson Correlation Coefficient (PEARCC)

The Pearson correlation coefficient $\rho_{X,Y}$ is one of several correlation coefficients that you need to choose between when you want to measure a correlation.

Idea: Measure how two variables move together (linearly).

$$\rho_{X,Y} = \frac{\operatorname{Cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{\mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]}{\sqrt{\mathbb{E}\left[X^2\right] - (\mathbb{E}[X])^2}\sqrt{\mathbb{E}\left[Y^2\right] - (\mathbb{E}[Y])^2}}$$

Cauchy–Schwarz inequality

$$X' = X - \mathbb{E}[X], Y' = Y - \mathbb{E}[Y]$$

$$|\mathbb{E}[X'Y']| \leq \sqrt{\mathbb{E}[X'^2]} \cdot \sqrt{\mathbb{E}[Y'^2]}$$

$$|\operatorname{Cov}(X,Y)| \leq \sigma_X \sigma_Y$$

Interpretation

- $\rho \approx +1$: strong positive linear relationship
- $\rho \approx -1$: strong negative linear relationship
- $\rho \approx 0$: no linear relationship

Partial Correlation Coefficient (PCC)

The Partial correlation coefficient measures the relationship between X_i and Y after removing the effect of other variables.

$$Z = \{X_k\}_{k \neq i}$$

$$X_i = \sum_{k \neq i} \beta_k X_k + e_{X_i} = \hat{X} + e_{X_i}$$

e_{X_i} = part of X_i not explained by Z

$\hat{X}_i$ = part of X_i explained by Z

$$Y = \sum_{k \neq i} \gamma_k X_k + e_Y = \hat{Y} + e_Y$$

e_Y = part of Y not explained by Z

$\hat{Y}$ = part of Y explained by Z

$$\rho_{X_i, Y | X_{-i}} = corr(e_{X_i}, e_Y)$$

In linear regression terms, this is defined as the correlation between the residuals (what remains after removing the linear effect of the other variables)!

by simplicity assume variables are centered → no intercept

Partial Correlation Coefficient (PCC)

Intuition with two variables

$$Z = X_2 \qquad X_1 = \beta_2 X_2 + e_{X_1} \qquad Y = \gamma_2 X_2 + e_Y$$

$$e_{X_1} = X_1 - \beta_2 X_2$$
$$e_Y = Y - \gamma_2 X_2$$

In linear regression:

$$\beta = \frac{\text{Cov}(X, Y)}{\text{Var}(X)}$$

how much X and Y move together

how much X varies

it measures how much Y changes per unit variation in X.

What is the regression doing?

Residual: $e = Y - \beta X$

We minimize the mean squared error to estimate β:

$$\begin{aligned}\mathbb{E}[(e)^2] &= \mathbb{E}[(Y - \beta X)^2] \\ &= \mathbb{E}[Y^2 - 2\beta XY + \beta^2 X^2] \\ &= \mathbb{E}[Y^2] - 2\beta\mathbb{E}[XY] + \beta^2\mathbb{E}[X^2]\end{aligned}$$

Take derivative w.r.t. β $\quad -2\mathbb{E}[XY] + 2\beta\mathbb{E}[X^2] = 0$

$$\beta = \frac{\mathbb{E}[XY]}{\mathbb{E}[X^2]} = \frac{\text{Cov}(X, Y)}{\text{Var}(X)}$$

We are assuming centered variables (no intercept)

Partial Correlation Coefficient (PCC)

Simple example with two variables

$$Z = X_2 \qquad X_1 = \beta_2 X_2 + e_{X_1} \qquad e_{X_1} = X_1 - \beta_2 X_2$$
$$Y = \gamma_2 X_2 + e_Y \qquad e_Y = Y - \gamma_2 X_2$$

$$\begin{aligned}\mathrm{Cov}(e_{X_1}, e_Y) &= \mathrm{Cov}(X_1 - \beta_2 X_2,\ Y - \gamma_2 X_2) \\ &= \mathrm{Cov}(X_1, Y) - \gamma_2 \mathrm{Cov}(X_1, X_2) - \beta_2 \mathrm{Cov}(X_2, Y) + \beta_2 \gamma_2 \mathrm{Var}(X_2)\end{aligned}$$

$$\beta_2 = \frac{\mathrm{Cov}(X_1, X_2)}{\mathrm{Var}(X_2)}, \quad \gamma_2 = \frac{\mathrm{Cov}(Y, X_2)}{\mathrm{Var}(X_2)}$$

$$\mathrm{Cov}(e_{X_1}, e_Y) = \mathrm{Cov}(X_1, Y) - \frac{\mathrm{Cov}(X_1, X_2)\mathrm{Cov}(Y, X_2)}{\mathrm{Var}(X_2)}$$

Partial Correlation Coefficient (PCC)

Simple example with two variables

$$Z = X_2 \qquad X_1 = \beta_2 X_2 + e_{X_1} \qquad e_{X_1} = X_1 - \beta_2 X_2$$

$$Y = \gamma_2 X_2 + e_Y \qquad e_Y = Y - \gamma_2 X_2$$

Compute variances of residuals

They are not independent

$$\begin{aligned}
\mathrm{Var}(e_{X_1}) &= \mathrm{Var}(X_1 - \beta_2 X_2) \\
&= \mathrm{Var}(X_1) + \beta_2^2 \mathrm{Var}(X_2) - 2\beta_2 \mathrm{Cov}(X_1, X_2) \\
&= \mathrm{Var}(X_1) + \left(\frac{\mathrm{Cov}(X_1, X_2)}{\mathrm{Var}(X_2)}\right)^2 \mathrm{Var}(X_2) - 2\left(\frac{\mathrm{Cov}(X_1, X_2)^2}{\mathrm{Var}(X_2)}\right) \\
&= \mathrm{Var}(X_1) - \frac{\mathrm{Cov}(X_1, X_2)^2}{\mathrm{Var}(X_2)} \\
&= \mathrm{Var}(X_1)\left(1 - \frac{\mathrm{Cov}(X_1, X_2)^2}{\mathrm{Var}(X_1)\mathrm{Var}(X_2)}\right) \\
&= \mathrm{Var}(X_1)\left(1 - \rho_{X_1 X_2}^2\right)
\end{aligned}$$

$$\beta_2 = \frac{\mathrm{Cov}(X_1, X_2)}{\mathrm{Var}(X_2)}$$

$$\mathrm{Var}(e_Y) = \mathrm{Var}(Y)\left(1 - \rho_{YX_2}^2\right)$$

Partial Correlation Coefficient (PCC)

Simple example with two variables

$$Z = X_2 \qquad X_1 = \beta_2 X_2 + e_{X_1} \qquad e_{X_1} = X_1 - \beta_2 X_2$$
$$Y = \gamma_2 X_2 + e_Y \qquad e_Y = Y - \gamma_2 X_2$$

$$\rho_{X_1 Y \cdot X_2} = \frac{\mathrm{Cov}(e_{X_1}, e_Y)}{\sqrt{\mathrm{Var}(e_{X_1})\mathrm{Var}(e_Y)}} = \frac{\mathrm{Cov}(X_1, Y) - \frac{\mathrm{Cov}_{(X_1, X_2)}\mathrm{Cov}_{(Y, X_2)}}{\mathrm{Var}_{(X_2)}}}{\sqrt{\mathrm{Var}(X_1)\left(1 - \rho^2_{X_1 X_2}\right)\mathrm{Var}(Y)\left(1 - \rho^2_{Y X_2}\right)}}$$

Substitute everything and simplify

Total correlation — *part explained through* X_2

$$\rho_{X_1 Y | X_2} = \frac{\rho_{X_1 Y} - \rho_{X_1 X_2}\rho_{Y X_2}}{\sqrt{(1 - \rho^2_{X_1 X_2})(1 - \rho^2_{Y X_2})}}$$

we rescale by the remaining variability, so we measure a pure correlation

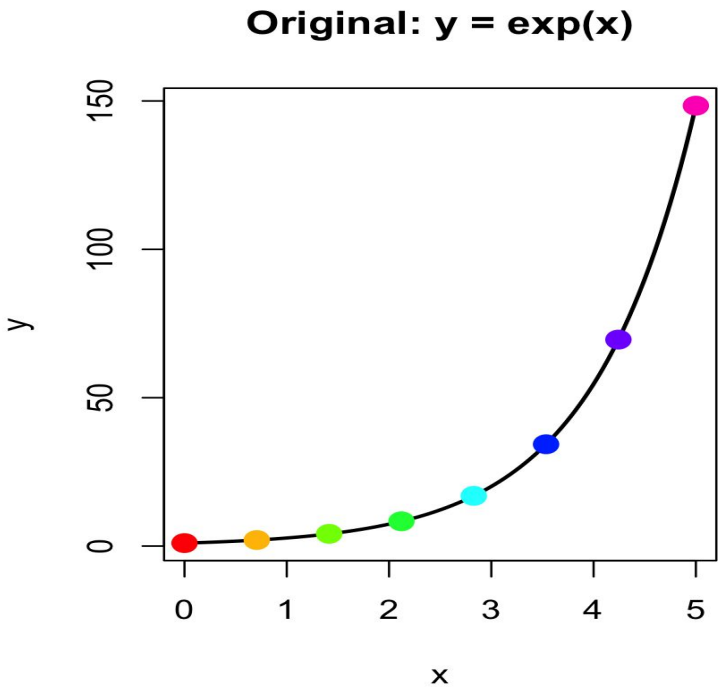
Partial Rank Correlation Coefficient (PRCC)

A global sensitivity analysis method used to determine the monotonic, nonlinear relationship between input parameters and model outcomes while keeping other parameters constant.

PRCC vs. PCC:

PCC only captures linear relationships

PRCC is robust against nonlinear, but monotonic, trends!



Partial Rank Correlation Coefficient (PRCC)

PRCC is just partial correlation, but computed on ranks instead of raw values.

The idea of PRCC:

"Keep everything the same, but replace values with **ranks**."

1. Start from your data

$(X_1^{(n)}, X_2^{(n)}, Y^{(n)})$

2. Apply rank transformation

$X_1 \to R(X_1), \quad X_2 \to R(X_2), \quad Y \to R(Y)$

Each value becomes its **position** (1, 2, …, N)

3. Do the SAME procedure as before

Regress $R(X_1)$ on $R(X_2)$ → residuals $e_{R(X_1)}$

Regress $R(Y)$ on $R(X_2)$ → residuals $e_{R(Y)}$

4. Compute correlation

$\text{PRCC}_{X_1} = \text{Corr}(e_{R(X_1)}, e_{R(Y)})$

Partial Rank Correlation Coefficient (PRCC)

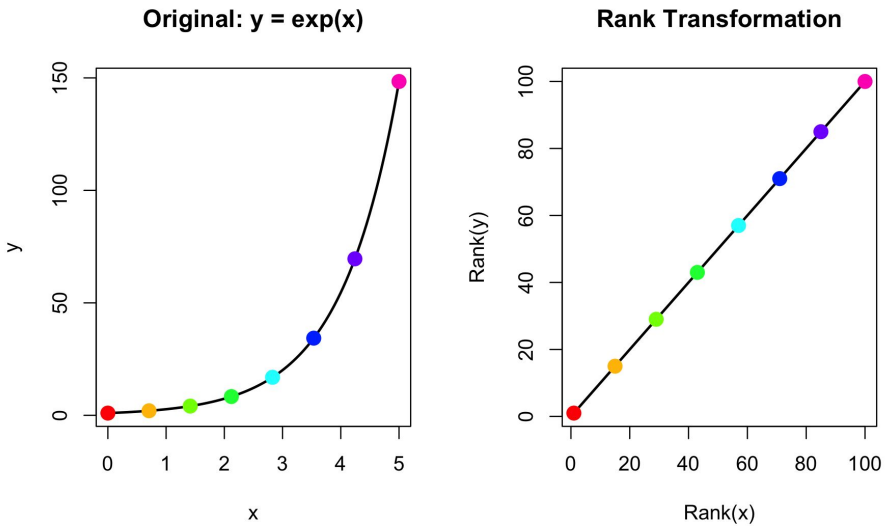
PRCC is just partial correlation, but computed on ranks instead of raw values.

The idea of PRCC:

"Keep everything the same, but replace values with **ranks**."

Ranking does two things:

1. Removes dependence on scale
2. Turns any monotonic relationship into a linear one in ranks



Correlation Coefficients

PCC → PC → PRCC
(from simple correlation → removing confounding → handling nonlinearity)

Pearson Correlation (PCC)
Measures linear relationship between X and Y

✅ Simple, fast
❌ Affected by other variables (confounding)
❌ Only detects linear relationships
“Do X and Y move together?”

Partial Correlation (PC)
Measures relationship between X and Y **controlling for other inputs**
Computed via correlation of residuals

✅ Removes confounding effects
❌ Still assumes linearity
“Do X and Y move together independently of others?”

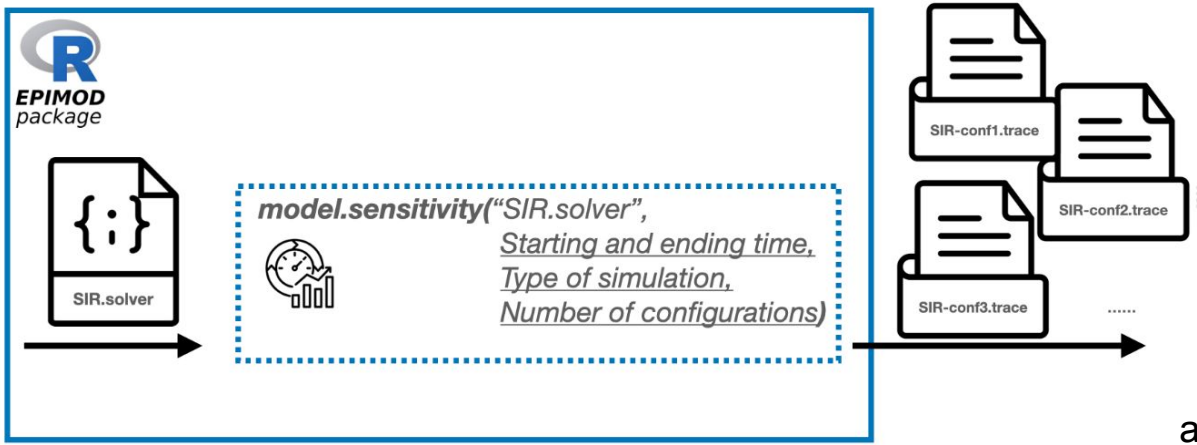
Partial Rank Correlation (PRCC)
Same as PC but on **ranked data**
Captures monotonic (not necessarily linear) relationships

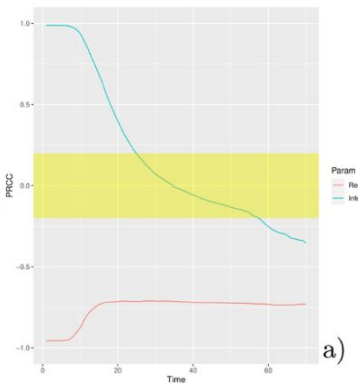
✅ Robust, widely used in global sensitivity analysis
❌ Fails for non-monotonic relationships
“Do X and Y move together consistently (increasing/decreasing), independently of others?”

Sobol indices
Variance-based → capture nonlinearities + interactions

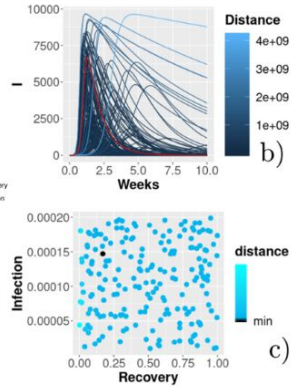
Model Sensitivity

model_sensitivity(): identify the set of parameter that impact the most on the model outcome (dynamics) with the Partial Rank Correlation Coefficients





a) Study the input parameters through the PRCC.



b-c) Study the parameter search space with respect to the the reference data

https://qbioturin.github.io/epimod/

Model Sensitivity

The simulations depend on the parameters (initial state and transition rates) set from GreatSPN

Change parameters from ***functions***

parameters_fname.csv

Describes what parameters vary without modifying the PNPRO file and how to define them

Tag & Meaning	Name	Value	Further Parameters
`i=` Complete **initial marking**	Usually `init`	Vector of numbers equal to the number of places or the name of a R function returning that vector	**input parameters** needed by the R function, which can be an R built-in function or a ***user-defined function***
`m=` Initial marking of a **specific place**	Name of the **place**	An integer number or the name of a R function returning a integer number	
`c=` **Constant rate** for a transition	Name of the **transition**	An real number or the name of a R function returning a real number	
`g=` Rate for a **general transition**	Name from **file in which will be saved the value**	An real number or the name of a R function returning a real number	

user-defined function included in the R script specified

functions_fname.R

Contains user-defined R functions used in `parameters_fname` and for analysis

Example	Purpose	Purpose	Notes
`init_generation`	Generates the **initial marking** values	`parameters_fname` (with tag `i`)	Initial conditions
`target`	Extracts the **target values** from model output (for PRCC calculation)	`target_value`	Selects one or more places to track over time.
`mse`	Computes the **error/ distance** between model output and reference data	`distance_measure`	For instance Mean Squared Error

Model Sensitivity

parameters_fname.csv

Describes what parameters vary without modifying the PNPRO file and how to define them

Tag & Meaning	Name	Value	Further Parameters
`i=` Complete **initial marking**	Usually `init`	Vector of numbers equal to the number of places or the name of a R function returning that vector	**input parameters** needed by the R function, which can be an R built-in function or a ***user-defined function***
`m=` Initial marking of a **specific place**	Name of the **place**	An integer number or the name of a R function returning a integer number	
`c=` **Constant rate** for a transition	Name of the **transition**	An real number or the name of a R function returning a real number	
`g=` Rate for a **general transition**	Name from **file in which will be saved the value**	An real number or the name of a R function returning a real number	

user-defined function included in the R script specified

functions_fname.R

Contains user-defined R functions used in `parameters_fname` and for analysis

Example	Purpose	Purpose	Notes
`init_generation`	Generates the **initial marking** values	`parameters_fname` (with tag `i`)	Initial conditions
`target`	Extracts the **target values** from model output (for PRCC calculation)	`target_value`	Selects one or more places to track over time.
`mse`	Computes the **error/ distance** between model output and reference data	`distance_measure`	For instance Mean Squared Error

```
1  i; init; init_generation; min_init = 10; max_init = 200
2  c; Recovery; runif; n=1; min = 0; max=0.1
3  c; Infection; runif; n=1; min = 0; max=0.1
4
```

```r
init_generation<-function(min_init , max_init)
{
  S=runif(n=1,min=min_init,max=max_init)
  # It returns a vector of lenght equal to 3 since the marking is
  # defined by the three places: S, I, and R.
  return( c(S, 1,0) )
}
```